BeliefBench

Domain-Independent Usage Guide

Custom Worlds, Exact Posteriors, LLM Measurements, and Case Studies

BeliefBench is a configurable probabilistic laboratory. The user defines the latent states, graph, conditional probability tables, evidence language, decisions, and utilities. The engine generates seeded scenarios, computes exact reference posteriors, measures model outputs, and produces auditable metrics and figures. No application domain or belief-state label is built into the engine.

Version 0.6.0
July 10, 2026

Contents

Figure 1: BeliefBench workflow. The exact reference posterior is retained by the evaluator and is not revealed in the prompt.

1 What BeliefBench is

Standard retrieval-augmented generation (RAG) can give an LLM relevant text, but it normally cannot tell an evaluator what probability a rational reasoner should assign to each possible latent state after reading that evidence. For example, a financial system may need probabilities for Bull, Bear, and Crisis regimes, while a diagnostic system may need probabilities for competing conditions. The retrieved documents may be incomplete, mutually dependent, stale, or contradictory, and their relationship to the true state is usually unknown. Consequently, ordinary RAG benchmarks can measure answer accuracy or citation quality, but they cannot cleanly determine whether an LLM updated its uncertainty correctly or whether a failure originated in retrieval, probabilistic reasoning, belief reporting, or decision-making.

BeliefBench supplies the missing experimental control. The researcher defines a small probabilistic world, samples a hidden state and observations from it, converts those observations into natural language, and computes the exact Bayesian posterior for precisely the information shown to the LLM. The model's response can then be compared with a known probabilistic reference rather than a subjective label. The goal is not to force an LLM to use a particular internal algorithm; it is to measure whether its observable beliefs and choices behave as they should under controlled uncertainty.

The principal measurements are:

- **Posterior accuracy:** how closely the reported probability distribution matches the exact posterior.
- **Calibration:** whether states assigned probability p occur approximately p of the time across generated scenarios.
- **Repeatability:** whether identical calls with the same evidence yield stable reported beliefs.
- **Evidence sensitivity:** whether beliefs move appropriately when evidence is removed, replaced, or contradicted.
- **Elicited–inferred agreement:** whether explicitly stated probabilities agree with beliefs inferred from payoff choices.
- **Decision consistency:** whether the selected action is optimal under the model's own reported belief and the declared utility matrix.

These properties matter whenever an LLM informs a consequential decision. Even when the model correctly identifies the most-probable latent state, its full probability distribution may still be severely overconfident, unstable across identical calls, inconsistent with its choices, or incompatible with its selected action. Correct classification is therefore necessary in many applications but is not sufficient evidence that the model exposes a trustworthy probabilistic belief state.

BeliefBench is domain-independent. It can represent equipment condition, diagnostic hypotheses, scientific theories, legal outcomes, weather regimes, or financial states. The Bull/Bear/Crisis world in Section 7 is a case study rather than a built-in ontology. The simulated model is only a software test double; scientific conclusions require a real-model run.

2 Installation and first run

```
python -m venv .venv
source .venv/bin/activate
python -m pip install https://github.com/mfrdixon/BeliefBench-Python/releases/download/v0.6.0/beliefbench
    -0.6.0-py3-none-any.whl
export OPENAI_API_KEY='your-key'
export OPENAI_MODEL='gpt-4.1-mini'
export BELIEFBENCH_API_URL='https://beliefbench-api.onrender.com'
export BELIEFBENCH_SERVICE_TOKEN='customer-access-token'
```

The installed package is a thin client: the proprietary benchmark engine remains on the hosted service. The user supplies an OpenAI API key because model charges and provider rate limits belong to the user's OpenAI account. The SDK reads `OPENAI_API_KEY` or accepts `openai_api_key=` explicitly, sends it only in the `X-OpenAI-API-Key` HTTPS request header, and never places it in YAML or result files. A separate BeliefBench service token authenticates access

to the hosted benchmark. Do not paste either secret into a notebook or commit it to source control.

Verify connectivity before submitting a job:

```
import os
from beliefbench import BeliefBench

with BeliefBench(os.environ["BELIEFBENCH_API_URL"]) as client:
    print(client.health())
    archive = client.run("my_world.yaml", "beliefbench-results.zip")
```

The health response publishes service limits. Each configuration must set `max_output_tokens`; the server applies that value to every OpenAI Responses API call and rejects values above its per-call ceiling. Before execution, it also computes the worst-case number of calls implied by scenarios, views, methods, repeats, inferred-belief menus, actions, and trajectories. Jobs whose worst-case output allocation exceeds the aggregate server token limit are rejected before any OpenAI call is made.

3 The configuration is the benchmark world

Belief states are read from the latent node's `outcomes`; they do not appear in engine code. The state order is consequential: it determines prior order, the latent axis of child CPTs, prompt order, output columns, and payoff-matrix columns.

A **conditional probability table (CPT)** lists the probability of every possible outcome of a node for every combination of its parents' values. A root node has no parents, so its CPT is simply its prior distribution. A child node's CPT describes how informative that observation is about its parents. For example, $P(\text{Positive Test} \mid \text{Present}) = 0.90$ means that a positive result occurs in 90% of worlds where the condition is present; it is not the posterior probability that the condition is present after observing a positive test. Bayes' rule combines that likelihood with the prior and all other observations to obtain the posterior.

3.1 Minimal two-state example

```
world:
  latent_node: Condition
  nodes:
    Condition:
      outcomes: [Absent, Present]
      cpt: [0.80, 0.20]
    Test:
      outcomes: [Negative, Positive]
      parents: [Condition]
      cpt:
        - [0.95, 0.05]   # P(Test | Absent)
        - [0.10, 0.90]   # P(Test | Present)
      templates:
        - "The test was negative."
        - "The test was positive."

decision:
  actions: [Do_Nothing, Investigate]
  payoffs:
    - [1.0, -3.0]
    - [-0.2, 1.0]
```

This example produces a binary JSON schema automatically, infers two-dimensional beliefs, and validates a 2×2 utility matrix. No changes to Python are required.

What the example is testing

The prior probability of `Present` is 0.20. A positive test is much more likely when the condition is present (0.90) than absent (0.05), so the exact posterior is

$$P(\text{Present} \mid \text{Positive}) = \frac{0.90(0.20)}{0.90(0.20) + 0.05(0.80)} = 0.8182.$$

The LLM is not being tested on whether it recognizes the word "positive." It is being tested on whether it combines the base rate with the test's sensitivity and false-positive rate, reports the resulting uncertainty, repeats that assessment reliably, and chooses an action consistent with the supplied payoffs.

An illustrative prompt and response are:

```
Prior
Absent 0.800000
Present 0.200000

Evidence
- The test was positive.

Task
Estimate posterior probabilities for: Absent, Present.
Return JSON only. Probabilities must sum to one.

LLM response
{"Absent": 0.22, "Present": 0.78}
```

The response has the correct direction but remains less confident than the exact posterior. BeliefBench records that difference instead of collapsing both vectors to the same top-label result.

Table 1: Illustrative output for the positive-test scenario.

Quantity	Absent	Present	Interpretation
Prior	0.8000	0.2000	Condition is initially uncommon.
Exact posterior	0.1818	0.8182	Bayesian reference after the positive test.
LLM response	0.2200	0.7800	Correct update, but modest under-confidence.

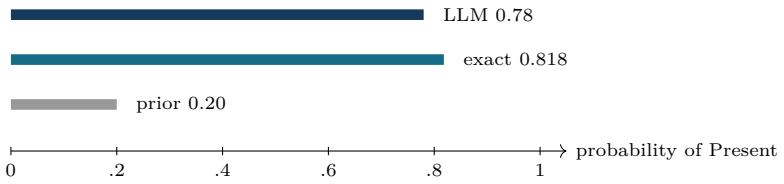

Figure 2: Worked-example output. The primary measurement is the distance between the LLM distribution and the exact posterior, not merely whether both select `Present`.

3.2 Node and CPT schema

Every node has a unique name and nonempty outcome list. A root node has no `parents`; its CPT is a probability vector. A child lists parents and a nested CPT whose axes are ordered as `parents` followed by the node outcome. Thus if parent cardinalities are $(k_1, \ldots, k_m)$ and the child has r outcomes, the required shape is $(k_1, \ldots, k_m, r)$. Every final-axis row must be nonnegative and sum to one.

Templates translate categorical outcomes into evidence sentences. There must be one template per outcome. If omitted, BeliefBench creates the neutral fallback "*Node was Outcome*." Templates should state observations without revealing likelihood ratios, the sampled state, or the posterior.

3.3 Graph validation

Before generation, BeliefBench topologically sorts the graph and rejects unknown parents or cycles. It then checks CPT dimensions, row normalization, latent-node existence, template cardinality, and decision-matrix dimensions. A valid graph can contain any number of discrete states and observable nodes, subject to the computational cost of exact inference.

4 Experiment controls

Key	Meaning
`seed`	Master seed for scenario generation and controlled mutations.
`scenarios`	Number of independently generated worlds.
`repeats`	Identical measurements per scenario, method, and evidence view.
`trajectory_length`	Number of observations displayed in trajectory diagnostics.
`llm_mode`	`simulated` for testing or `api` for empirical evaluation.
`model`	Versioned model identifier passed to the API adapter.
`partial_omit`	Number of observations removed from the partial view.
`standard_rag_top_k`	Number of documents returned by the similarity baseline.
`entropy_bins`	Boundaries for low, medium, and high normalized entropy.
`dataset_name`	Subdirectory used for generated scenario records.
`output_dir`	Result directory relative to the repository root.

5 Evidence views and comparison methods

For each sampled assignment, the reference view contains all observations, the partial view omits configured observations, and the noisy view changes one observation to a different valid outcome. Exact inference is rerun on the presented view. This produces both a full posterior b^{full} and view posterior b^v.

The method labeled BeliefBench receives the natural-language rendering of the controlled view. BaselineRAG performs deterministic similarity retrieval over the template corpus and distractor documents. The latter is a diagnostic baseline, not a competitive claim about production retrieval systems. Strong empirical work should add BM25, dense, hybrid, and reranked baselines under matched context budgets.

Retrieval mismatch is $D_{JS}(b^v, b^{\text{full}})$, and conditional reasoning/elicitation error is $D_{JS}(q^v, b^v)$. Total error is $D_{JS}(q^v, b^{\text{full}})$. These are descriptive components; JS divergence is not additive.

6 Beliefs and decisions

The belief prompt contains the configured prior, rendered evidence, and user-defined state names. The API JSON schema is generated from those names. State labels must therefore be unique strings suitable as JSON property names. Responses are normalized after parsing.

Inferred beliefs are estimated from binary payoff menus without requesting probabilities. The menu dimension is derived from the number of states. This channel provides convergent evidence but is not direct access to an internal state. The decision prompt uses user-defined actions and payoffs; consistency compares the chosen action with the expected-utility optimum under the elicited belief, while reference agreement and regret use the exact posterior.

7 Financial case study: Bull, Bear, and Crisis

The file `configs/financial_case_study.yaml` asks a concrete question: *given a prior market outlook and a controlled set of macro-financial observations, does the LLM express an appropriate probability distribution over Bull, Bear, and Crisis regimes, and does it act consistently with that distribution?* This is more demanding than asking for a market label. Portfolio risk depends on the entire distribution: a 55% Bull, 40% Bear, 5% Crisis assessment can imply a different position from a 95% Bull, 4% Bear, 1% Crisis assessment even though both have the same top state.

The case-study prior is $(0.50, 0.30, 0.20)$. Earnings, Credit, Funding, and Activity depend on the latent market state; Volatility depends jointly on Market State and Credit. The CPTs define the controlled statistical meaning of observations such as weak earnings or wide credit spreads. Long, Neutral, and Cash are possible actions, and their payoff rows make the consequences of tail risk explicit. These probabilities and utilities are experimental parameters,

not estimates of a real market.

Run the hosted case study through the installed SDK; source execution is neither required nor supported for users:

```
import os
from beliefbench import BeliefBench

with BeliefBench(
    openai_api_key=os.environ["OPENAI_API_KEY"],
    service_token=os.environ["BELIEFBENCH_SERVICE_TOKEN"],
) as client:
    print(client.health())
    archive = client.run(
        "financial_full.yaml",
        "financial-full-results.zip",
        client_request_id="financial-study-001",
    )
print(f"Saved {archive}")
```

The SDK uploads the declared YAML to the private hosted engine and downloads a ZIP containing the sanitized configuration, manifest, generated scenarios, tables, and figures. To adapt the case study, copy `financial_full.yaml`, revise the latent states, all affected CPT axes, evidence templates, actions, and utilities, then submit the new file with `client.run`. Do not reinterpret the financial CPTs as validated market estimates; they are controlled experimental parameters. For a confirmatory study, use multiple independent seeds, balanced entropy strata, repeated calls, scenario-clustered confidence intervals, and the resumable multi-seed SDK example distributed with the client repository.

7.1 Example prompt and measured output

One generated prompt can take the following form:

```
Prior
Bull 0.500000
Bear 0.300000
Crisis 0.200000

Evidence
- Corporate earnings weakened.
- Credit spreads widened.
- Funding conditions were stressed.
- Activity indicated contraction.
- Market volatility was extreme.

Task
Estimate posterior probabilities for: Bull, Bear, Crisis.
Return JSON only. Probabilities must sum to one.

Example LLM output
{"Bull": 0.01, "Bear": 0.14, "Crisis": 0.85}
```

The important question is not simply whether the model says "Crisis." BeliefBench measures how close $(0.01, 0.14, 0.85)$ is to the exact posterior implied by the configured likelihoods, whether the response remains stable across identical calls, whether confidence changes appropriately if the Funding observation is omitted or contradicted, whether payoff-menu choices imply a similar latent distribution, and whether the selected portfolio action is optimal under the reported probabilities.

7.2 Case-study summary

The completed API study evaluated `gpt-4.1-mini` over four independent seeds and 120 scenarios, balanced exactly across low-, medium-, and high-entropy strata. Each condition used five elicitation repeats; inferred beliefs and decisions were measured once per condition; BeliefBench used reference, partial, and noisy evidence views; and the diagnostic similarity-RAG baseline used the reference view only. The analysis contained 2,400 elicitation rows and 480 decision rows and used 2,000 bootstrap replications clustered by seed and scenario. Table 3 uses the matched reference

view for the method comparison, so differences are not caused by unequal information sets.

Table 3: Empirical GPT-4.1-mini financial-case results on the matched reference view. Lower is better except reference-action agreement.

Method	JS error	Inferred JS	Brier	Repeat. var.	Action agree.	Regret
BeliefBench	0.0789	0.1370	0.1650	0.00207	0.700	0.0496
BaselineRAG	0.1238	0.1548	0.2136	0.00312	0.450	0.1812

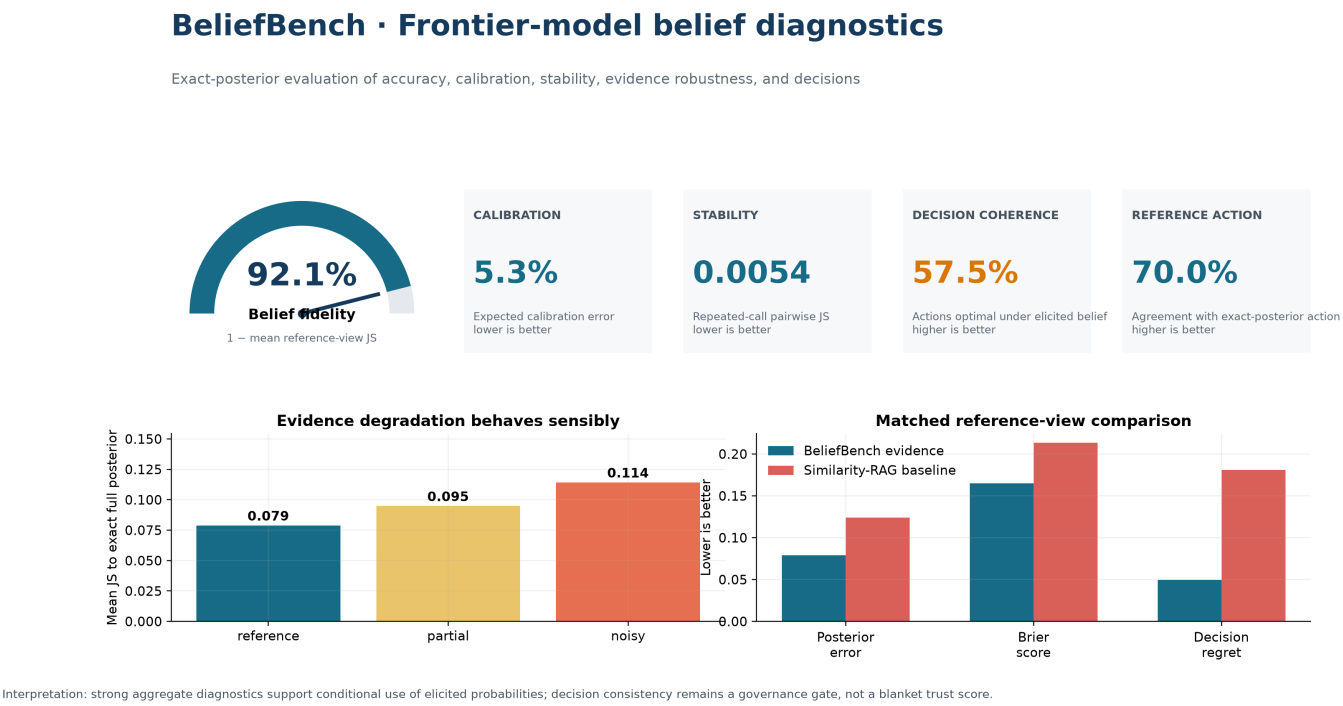

Figure 3: BeliefBench empirical summary dashboard for GPT-4.1-mini. The 92.1% dial is the transparent statistic 1−mean reference-view JS divergence, not a universal trust score. Calibration and stability are encouraging, evidence degradation is ordered sensibly, and the matched baseline comparison favors controlled evidence. Decision coherence of 57.5% remains a material governance warning.

The absolute diagnostics are more important than merely beating the baseline. Reference-view mean JS error was 0.0789, expected calibration error was 0.0533, and repeated-call pairwise JS instability was 0.00536. Error increased from 0.0789 under complete evidence to 0.0949 under partial evidence and 0.1143 under noisy evidence, indicating sensible degradation. The model selected the same most-probable state as the exact posterior in 67.3% of reference-view observations. Elicited and payoff-inferred beliefs differed by mean JS 0.0563, providing useful but incomplete convergent validity.

Matched scenario-clustered bootstrap comparisons favored the controlled evidence path: the BeliefBench-minus-baseline difference was -0.0449 for JS error (95% CI $[-0.0582, -0.0326]$), -0.0485 for Brier score (95% CI $[-0.0638, -0.0327]$), and -0.1316 for regret (95% CI $[-0.1803, -0.0854]$). Reference-action agreement improved by 25 percentage points (95% CI $[15, 35]$). These results establish measurable value relative to the diagnostic retriever, but they do not erase the absolute limitations of the tested model.

A user should regard the agent as passing this case study only if prespecified acceptance thresholds are met with scenario-clustered confidence intervals across seeds and entropy strata. Here, action consistency under the model's own elicited belief was only 57.5%, even though reference-action agreement reached 70.0%. The natural deployment pattern is therefore to use elicited probabilities as monitored inputs and compute actions in deterministic, validated downstream code. A low mean error alone is insufficient: severe tail failures, poor crisis calibration, instability, elicitation–choice disagreement, or high decision regret can each invalidate use in a financial workflow.

8 Running an API experiment

```
from beliefbench import BeliefBench

with BeliefBench() as client:  # reads OPENAI_API_KEY
    result = client.run(
        "my_world.yaml",
        "beliefbench-results.zip",
        client_request_id="registered-run-001",
    )
```

In the YAML, set `llm_mode: api` and a conservative `max_output_tokens`. SDK v0.6.0 uses `gpt-4.1-mini` by default to control cost. A user can override it with the `openai_model=` constructor argument, the `OPENAI_MODEL` environment variable, or the YAML `model` key; precedence follows that order. The hosted service creates a background job, polls its status, displays measurement and trajectory progress, and downloads the completed ZIP containing the sanitized configuration, provenance manifest, tables, and plots. It does not return or store the OpenAI key. The complete executable example is in `notebooks/HostedAPIWalkthrough.ipynb`.

Current provider scope. The hosted release currently supports GPT frontier models available through the OpenAI Responses API only. It does not yet provide adapters for Anthropic, Google, open-weight endpoints, or locally hosted models. Results therefore characterize the exact OpenAI model identifier and API behavior recorded in the manifest; they should not be generalized to another provider, model snapshot, or deployment.

Before a scientific run, freeze the service-engine version and configuration; archive raw prompts and responses under the approved retention policy; record the model identifier, date, region, decoding settings, token ceilings, and failures; preregister primary metrics; balance entropy support; and cluster uncertainty intervals by scenario. Never present simulated-adapter output as empirical LLM evidence.

9 Outputs and interpretation

Artifact	Contents
`summary.csv`	Method-level JS error, inferred error, proper scores, ECE, stability, decision agreement, and regret.
`belief_accuracy.csv`	Scenario/repeat/view rows with probability, posterior, retrieval, and reasoning fields.
`repeatability.csv`	Component variance and mean pairwise JS across repeated calls.
`decision_consistency.csv`	Chosen, belief-optimal, and reference-optimal actions plus regret.
`entropy_analysis.csv`	Reference entropy and repeated-call instability.
`belief_trajectories.csv`	Reference and measured state probabilities by step.
`plots/`	Posterior error, calibration, repeatability, entropy, information gain, decisions, trajectories, and simplex.

Probability columns are indexed as `p_0`, `p_1`, and so forth because labels can be arbitrary; their meaning follows the configured state order, which is also stored with generated scenarios. The simplex plot is meaningful only for three states. For other cardinalities, BeliefBench emits an explanatory placeholder while all other plots remain available.

10 Creating a defensible custom benchmark

This section is a procedure for researchers creating a new benchmark world.

1. **Write the measurement question.** State which LLM property you want to measure—for example posterior accuracy under conflicting diagnostic evidence—before writing scenarios.
2. **Define the latent states.** Make them mutually exclusive, collectively exhaustive for the experiment, and operationally interpretable. Avoid overlapping labels such as “moderate” and “uncertain.”
3. **Choose and justify the prior.** Record whether it comes from data, expert elicitation, or a deliberate stress-test design.
4. **Draw the DAG.** Add observations and dependencies before assigning probabilities. Do not represent duplicated or causally dependent evidence as independent children merely for convenience.

5. **Specify CPTs.** Populate every parent configuration, normalize every row, document the source of each probability, and inspect likelihood ratios for plausibility.
6. **Write evidence templates.** Each sentence should convey its categorical observation without naming the latent state, posterior, or likelihood ratio.
7. **Define actions and utilities separately.** Utilities express consequences and risk preferences; they should not be adjusted after seeing model behavior.
8. **Validate before calling an LLM.** Run graph validation, prior-predictive simulation, posterior spot checks, and sensitivity analyses over priors and CPTs.
9. **Predefine interventions and estimands.** Decide which observations may be omitted or corrupted and identify primary metrics, entropy strata, repeat count, and uncertainty intervals.
10. **Freeze and report the world.** Treat the complete YAML and its hash as part of the scientific instrument, not as an incidental settings file.

11 Troubleshooting

Use the following sequence; fix the first failing stage before moving to the next.

1. **Confirm the configuration loads.** YAML indentation must be consistent and node names must be unique.
2. **Resolve graph errors.** For "cycle or unknown parent," verify spelling and make every edge point from parent to child.
3. **Resolve CPT shape errors.** List parent cardinalities in the declared order. The required array shape is those cardinalities followed by the child's cardinality.
4. **Resolve normalization errors.** Each innermost CPT row must be nonnegative and sum to one within numerical tolerance.
5. **Resolve decision errors.** Supply one payoff row per action and one column per latent state, using the configured state order.
6. **Run simulated mode first.** If generation and plots succeed in simulated mode, configuration and output problems are separated from provider failures.
7. **Diagnose API failures from raw records.** Check authentication, model access, rate limits, refusal text, and JSON parsing separately. Use unique, concise state labels.
8. **Inspect sampling support.** If high-entropy cases are sparse, redesign CPTs or use a declared accept–reject sampler; do not infer an entropy effect from a handful of cases.
9. **Locate outputs.** The configured output directory is resolved from the repository root inferred from the configuration file's location.

12 Reproducibility checklist

An auditor should be able to reconstruct every reference posterior and trace every reported statistic to a model response. Use this audit sequence.

1. **Identify the implementation.** Record the BeliefBench version, repository commit or service-engine version, Python environment, dependency lock, and execution date.
2. **Verify the probabilistic world.** Archive the exact YAML, its cryptographic hash, latent-state order, graph, CPTs, templates, actions, and payoff matrix.
3. **Verify randomness.** Record the master seed, seed-spawning rule, scenario count, view mutations, repeat count, and generated scenario file.
4. **Verify exact references.** Recompute a sample of full and view-specific posteriors from the stored evidence and confirm normalization and agreement with exported values.
5. **Verify model provenance.** Record provider, full model identifier, API date, region if relevant, temperature, token limits, system prompt, and other decoding controls.
6. **Verify the response trail.** Retain raw prompts and responses, or approved cryptographic hashes with controlled access. Report refusals, parse failures, retries, and all exclusions.
7. **Verify analysis.** Archive metric definitions, state-column mapping, clustering unit, confidence-interval method, multiplicity policy, aggregation code, and plotting code.
8. **Verify claims.** Label results as simulated validation, local-model evidence, or remote-API evidence. Ensure captions and conclusions do not generalize beyond the tested graph, information views, and model version.

13 Scope and limitations

What the benchmark tells the user

The practical utility of BeliefBench is to turn an otherwise vague claim—"the model seems appropriately uncertain"—into a collection of falsifiable measurements against an exact probabilistic reference. For the tested OpenAI frontier model, prompt template, evidence language, graph, and date, the benchmark estimates whether elicited probabilities are close to the posterior warranted by the presented information; whether those probabilities are calibrated over repeated generated worlds; whether the same information produces stable reports; whether reports change in the right direction when information is removed or corrupted; whether payoff choices imply a compatible latent distribution; and whether actions are coherent with both the elicited belief and the declared utility function. This is evidence about the quality of the agent's *observable belief interface*, not proof that the model contains a literal Bayesian posterior internally.

That distinction determines how results should be used. If a model meets prespecified thresholds across seeds, entropy strata, information views, and decision regimes, a developer has evidence that its probability reports are usable as a monitored input to downstream software within the tested domain. Those reports can support abstention rules, risk tiers, human-review triggers, expected-utility calculations, scenario weighting, confidence-aware retrieval, portfolio limits, or escalation when elicited and inferred beliefs diverge. Belief trajectories can also be monitored in production for unexpected jumps, excessive entropy, or instability, provided production inputs remain sufficiently close to the benchmark's declared scope.

What a good result does—and does not—justify

A good benchmark result increases warranted confidence; it does not create unconditional trust. The user may trust that the tested model exhibited accurate, calibrated, stable, and decision-coherent probability reports under the benchmark's interventions, within the uncertainty of the reported estimates. The user may not infer that every individual probability is correct, that hidden chain-of-thought is Bayesian, that the model will remain calibrated after a provider update, or that performance transfers automatically to open-world evidence, adversarial prompts, new state definitions, or materially different priors. Trust must therefore be conditional, versioned, and bounded by acceptance criteria.

Before operational use, define metric-specific gates rather than a single leaderboard score. Examples include an upper confidence bound on JS divergence, maximum ECE, minimum repeatability, bounded crisis-state log loss, minimum elicited–inferred agreement, and maximum expected regret. Examine worst-case and high-entropy strata in addition to averages. If a gate fails, the appropriate response may be recalibration, prompt redesign, retrieval repair, narrower deployment, mandatory human review, or rejection of probabilistic outputs for that use case. If gates pass, continue shadow evaluation and periodically rerun the frozen benchmark because frontier-model behavior can change.

Scientific and operational limitations

BeliefBench provides an exact answer to a deliberately bounded question: *what posterior follows from this declared graph, these CPTs, and this information set?* Exact computation does not make the configured world true. Researchers remain responsible for defending the state definitions, graph structure, prior, likelihoods, evidence language, and utilities. A poorly specified model yields a precisely calculated but scientifically unhelpful reference.

The framework is most useful for controlled measurement and mechanism studies. It does not reproduce every feature of an open world: continuous variables may be discretized, evidence can be dependent in ways omitted by the DAG, regimes may change over time, and an LLM may use relevant background knowledge not represented in the prompt. Exact variable elimination can also become expensive as a graph's treewidth and outcome cardinalities grow.

Elicited probabilities and behaviorally inferred probabilities are observable measurements, not direct readings of an internal mental state. Prompt wording, rounding, decoding, refusal policies, and the identification model for payoff choices can all affect them. Agreement across elicitation, repeated choices, and actions strengthens measurement validity, while disagreement should be reported rather than hidden.

Finally, the included similarity baseline is diagnostic rather than state of the art. Calibration requires enough observations throughout the probability range; entropy effects require balanced uncertainty support; and repeated calls and evidence views are clustered within scenario. A defensible study therefore uses stronger retrieval baselines where appropriate, scenario-aware uncertainty intervals, versioned API provenance, sensitivity analysis, and claims restricted to the tested benchmark world. BeliefBench exposes these assumptions so they can be audited; it cannot remove them.